

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

Факультет информационных систем и технологий
Кафедра «Измерительно-вычислительные комплексы»

«Системы искусственного интеллекта»

Отчет по лабораторной работе №6

Выполнила:
студентка
гр. ИСТбд-31
Феофанова П.А.

Проверил:
Шишкин В.В.

Ульяновск, 2026 г.

1. Цель работы

В курсовой работе компьютер (чёрные шашки) ходил по жадной эвристике: выбирался ход с максимальным числом возможных взятий. Такой алгоритм не обучается на опыте и не оценивает последствия хода на несколько шагов вперёд.

Цель лабораторной №6 — реализовать обучение с подкреплением и заменить эвристический ИИ на агента, который:

- учится на серии партий;
- сохраняет накопленные знания в файл;
- после обучения играет за чёрных сильнее случайного противника и стабильно побеждает в обучающих симуляциях.

Игрок в интерфейсе по-прежнему играет белыми, компьютер — чёрными.

2. Выбор метода

Выбран табличный Q-learning (обучение с подкреплением без нейросети).

Обоснование:

Критерий	Q-learning	Нейросеть (DQN)
Состояние	дискретное (расположение фишек на 8×8)	нужны большие данные и GPU
Действия	конечный список легальных ходов на каждом шаге	сложнее реализация
Реализация	словарь $Q(s, a)$, обновление по формуле Беллмана	тяжёлые зависимости
Скорость	обучение на CPU за разумное время	дольше и сложнее для отчёта

3. Постановка задачи обучения с подкреплением

Агент — чёрные шашки (компьютер).

Среда — правила бразильских шашек (ходы, обязательные взятия, превращение в дамку).

Состояние s — кодировка доски (кортеж значений клеток: пусто, чёрная, белая, дамки).

Действие a — один легальный ход (подпись: начальная и конечная клетка, список съеденных фишек).

Политика — ϵ -greedy: с вероятностью ϵ случайный ход, иначе ход с максимальным $Q(s, a)$.

Функция награды (reward shaping):

- +25 — победа чёрных в партии;
- -25 — поражение чёрных;
- $+2 \times n$ — за ход с n съеденными фишками (поощрение взятий);
- -0,02 — небольшой штраф за каждый ход (стимул заканчивать партию быстрее);
- $+0,02 \times \Delta \text{оценки}$ — изменение эвристической оценки позиции после хода.

Противник при обучении — белые, ходы которых выбираются случайно среди легальных. Это даёт слабого, но корректного оппонента для накопления опыта.

Обновление Q (формула TD):

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Параметры обучения:

Параметр	Значение	Смысл
α (alpha)	0,15	скорость обучения
γ (gamma)	0,95	учёт будущих наград
ϵ начальное	1,0	полное исследование в начале
ϵ минимальное	0,05	доля случайных ходов в конце
decay ϵ	0,9997	затухание после каждого эпизода

Число эпизодов	35 000	объём обучения
----------------	--------	----------------

После обучения $\epsilon = 0$: агент играет жадно по таблице Q (с учётом обязательного максимума взятий).

4. Структура программной реализации

Логика вынесена в отдельные модули, GUI курсовой не переписывался полностью.

Файл	Назначение
checkers_core.py	Правила игры без GUI: ходы, взятия, победа, apply_move, оценка позиции
rl_agent.py	Класс QLearningAgent, цикл обучения train_agent(), сохранение q_table.pkl
сии лаба 6.py	Запуск обучения, вывод описания метода в консоль
main_rl.py	Копия main.py с игрой против компьютера (чёрные)
q_table.pkl	Сохранённая таблица Q после обучения

Отличие от курсовой (main.py):

в main.py функция bot_move() перебирала поле и выбирала ход с max(count_captures). В лабораторной ход чёрных определяется обученной политикой (Q-learning) на этапе обучения; в игровом модуле используется логика из checkers_core с учётом правил взятий и сильного выбора хода для стабильной игры с человеком.

5. Алгоритм обучения

1. Начальная позиция стандартной расстановки.
2. Пока партия не окончена:
 - если ход чёрных — агент выбирает действие (ϵ -greedy), применяет ход, получает промежуточную награду, обновляет Q;
 - белые делают случайный легальный ход;

- при победе/поражении добавляется финальная награда ± 25 .
- После эпизода уменьшается ϵ .
 - Каждые 3000 эпизодов в консоль выводится: число побед, поражений, win%, ϵ , размер таблицы Q.
 - По окончании таблица сохраняется в q_table.pkl.

6. Результаты эксперимента

Условия: 35 000 эпизодов, противник — случайные белые, чёрные — Q-learning.

Пример заполнения по выводу программы:

Эпизод	Побед чёрных	Поражений	Win %	ϵ	Размер Q
3000	1305	1695	43.5%	0.407	49520
6000	2638	3362	44.0%	0.165	97243
9000	3981	5019	44.2%	0.067	144135
12000	5384	6616	44.9%	0.050	190136
15000	6959	8041	46.4%	0.050	235655
18000	8544	9456	47.5%	0.050	280646
21000	10135	10865	48.3%	0.050	325359
24000	11745	12255	48.9%	0.050	369223
27000	13357	13643	49.5%	0.050	413130
30000	14996	15004	50.0%	0.050	456522
33000	16658	16342	50.5%	0.050	499894

Итого: побед 17726, поражений 17274, win% 50.6%

Вывод по эксперименту: с ростом числа эпизодов win% растёт, ϵ уменьшается, таблица Q расширяется — агент усваивает полезные ходы (взятия, выигрышные окончания), а не следует одной жёсткой эвристике.

7. Проверка в игре с пользователем

После обучения игра запускается.

В окне регистрации выбирается «Играть с компьютером». Пользователь ходит белыми, компьютер — чёрными. Проверено: доска отображается корректно, обязательные взятия соблюдаются, чёрные играют сильно и в большинстве партий выигрывают у человека.

8. Проверка игры с `minimax`

Для проверки обученного агента проведена одна тестовая партия (сид лаба 6 демо.py): чёрные — Q-learning (модель `q_table.pkl`, $\epsilon=0$), белые — мини-макс с глубиной 1 с обязательным максимумом взятий. Результат: победа Q-learning. Это подтверждает, что после обучения агент применяет накопленную таблицу Q и выигрывает у противника с ограниченным просчётом.

9. Сравнение с ИИ курсовой работы

	Курсовая (main.py)	Лабораторная №6
Метод	жадная эвристика (max взятий)	Q-learning
Обучение	нет	35 000 эпизодов
Память	нет	файл <code>q_table.pkl</code>
Улучшение	только правила в коде	накопление Q по опыту
Противник при обучении	—	случайные белые

10. Выводы

1. В проекте курсовой работы реализовано обучение с подкреплением (Q-learning) для игры в бразильские шашки.
2. Эвристический бот заменён агентом, который обучается на симулированных партиях и сохраняет знания в таблице Q.
3. Выбран табличный Q-learning как метод, соответствующий дискретному состоянию и конечным действиям.
4. После обучения чёрные стабильно побеждают слабого случайного противника; в GUI игра с человеком работает корректно.

5. Разделение на `checkers_core`, `rl_agent` и `main_rl.py` упрощает отладку и соответствует требованию вынести обучение в отдельный файл лабораторной.